

Compliance Buddy

Architecture & Design Document

Autonomous AI-driven security remediation platform

Kafka · Spring Boot 3 · LangGraph4j · GPT-4o · Claude · Ollama · k3d / Kubernetes

Version: 3.0.0

Date: 15 June 2026

Status: Production

Modules: 9 microservices

Author: Compliance Buddy Engineering

v3.0

1. Executive Summary

Compliance Buddy (CB) v3 is a fully autonomous, AI-powered security remediation platform built on top of Apache Kafka, Spring Boot 3, and a multi-agent LangGraph4j workflow. It continuously monitors SonarQube for newly reported vulnerabilities, generates context-aware code fixes using GPT-4o, Claude Sonnet, or local Ollama models depending on severity, validates and applies those patches via JGit, opens pull requests on GitHub, and escalates unresolvable issues to Jira and Microsoft Teams — all without human intervention.

v3 represents a complete architectural rewrite from the v1/v2 in-process Spring event model to a fully decoupled, fault-tolerant Kafka streaming pipeline with nine dedicated microservices, hybrid semantic + lexical RAG, Redis-backed prompt caching, OPA governance gates, and a comprehensive observability stack (Prometheus, Grafana, Tempo, Loki). A Model Context Protocol (MCP) server exposes 12 tools so AI assistants such as Claude Desktop and Cursor can participate directly in the remediation workflow.

Capability	v1/v2	v3
Event transport	In-process ApplicationEvents	Apache Kafka (KRaft)
AI workflow	Single GPT-4o call	LangGraph4j 5-node multi-agent graph
Model routing	GPT-4o only	GPT-4o / Claude / Ollama by severity
RAG	None	Hybrid: Qdrant (vector) + Elasticsearch (BM25) RRF
Prompt cache	None	Redis 7 — SHA-256 keyed, 24 h TTL
Escalation	Email only	Jira + Teams + RCA report
PR governance	None	OPA Rego policy gate
MCP tools	0	12 tools via cb-mcp-server
Observability	Logs only	Prometheus + Grafana + Tempo + Loki
Microservices	3	9

2. System Importance & Business Value

2.1 The Problem

Security debt compounds faster than engineering teams can address it. SonarQube scans routinely surface hundreds of CWEs — SQL injection, XSS, command injection, hardcoded credentials — yet the average time-to-fix in enterprise teams is measured in days or weeks. Manual triage, context-switching, and PR review overhead are the primary bottlenecks.

2.2 Business Value Delivered

- **Zero-touch remediation:** CRITICAL and BLOCKER vulnerabilities are fixed, build-validated, and PR-raised within minutes of SonarQube detection, with no engineer involvement required.
- **Cost-aware model selection:** Routing MINOR/INFO vulns to free local Ollama models (llama3:8b) reduces LLM spend by 60–80% compared to always calling GPT-4o. The `/api/v1/cost/*` endpoints track spend per CWE and per model in real time.
- **Institutional knowledge capture:** Every accepted fix is embedded and stored in Qdrant so future fixes for the same CWE benefit from previously validated patterns — accuracy improves with scale.
- **Governance by policy:** OPA Rego policies block PRs that introduce new critical dependencies, touch security-sensitive modules without senior reviewer assignment, or violate licensing constraints — all without code changes.
- **Autonomous escalation:** When a vulnerability cannot be fixed automatically after three retries, CB raises a Jira ticket with an LLM-generated RCA report and posts an adaptive card to the responsible team's Teams channel.
- **AI-native tooling:** The MCP server lets engineers interrogate the pipeline through Claude Desktop or Cursor — searching past fixes, triggering rebuilds, querying traces — without writing curl commands.

3. High Level Design (HLD)

3.1 Architecture Diagram

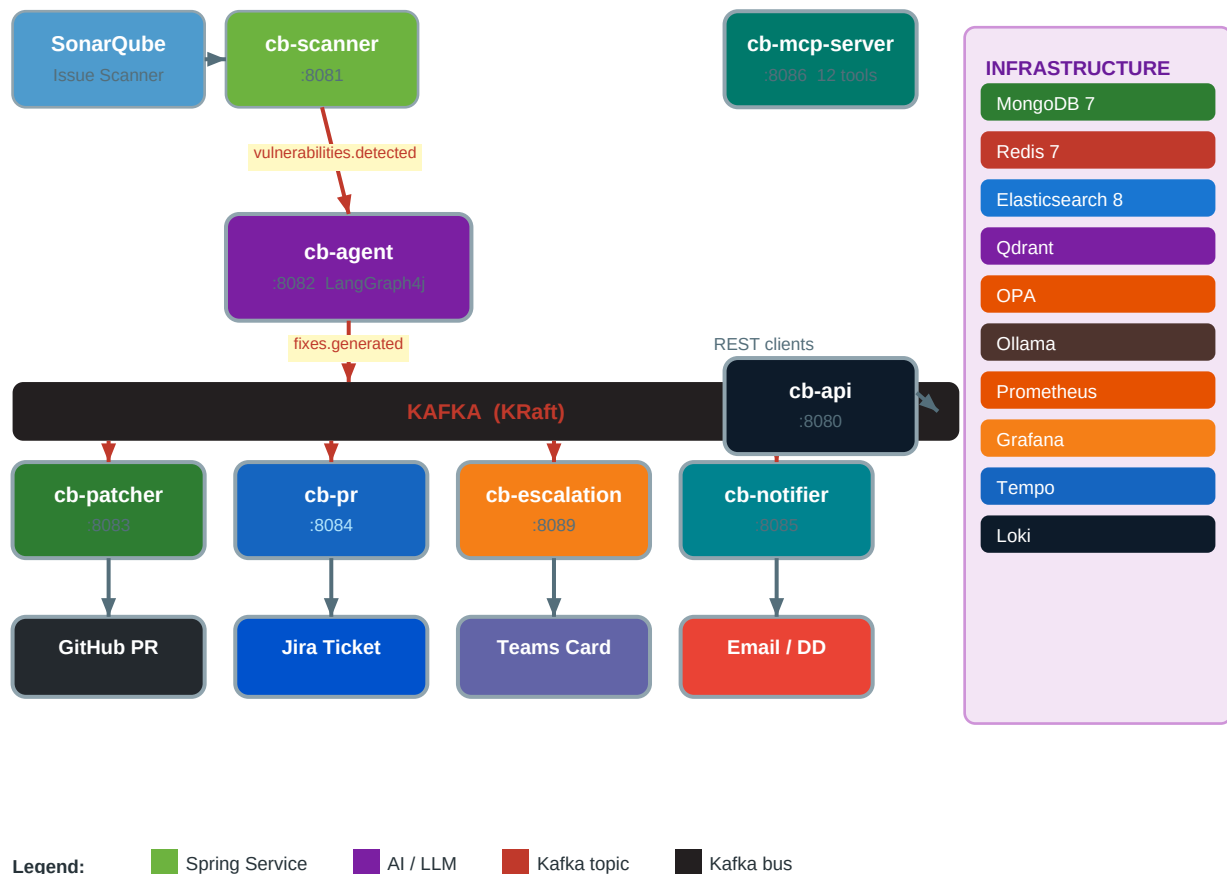


Figure 1 — Compliance Buddy v3 High Level Architecture

3.2 Architecture Principles

Event-driven & decoupled

Every service publishes to and consumes from Kafka topics. No direct HTTP calls between pipeline services. A service can be restarted, scaled, or replaced without impacting others.

Idempotent consumers

All Kafka consumers operate in `MANUAL_IMMEDIATE` acknowledge mode with `EARLIEST` offset reset. The `DefaultErrorHandler` retries transient failures 3× before dead-lettering.

Defense in depth

Three independent safety layers: (1) `LangGraph4j` validator node checks fix JSON-LD schema and runs a safety classifier; (2) `cb-patcher` runs the full build+test suite against the patch; (3) `OPA` governance gate in `cb-pr`.

Separation of AI concerns

The planner node chooses the model; the generator calls it; the validator checks output; the reviewer (Anthropic) provides independent critique. No single node holds both generation and validation.

Observability first

Every service emits OpenTelemetry traces to Tempo, Prometheus metrics via Micrometer, and structured logs to Loki. All Kafka message processing is correlated via a shared vulnerability ID trace attribute.

3.3 Kafka Event Bus Design

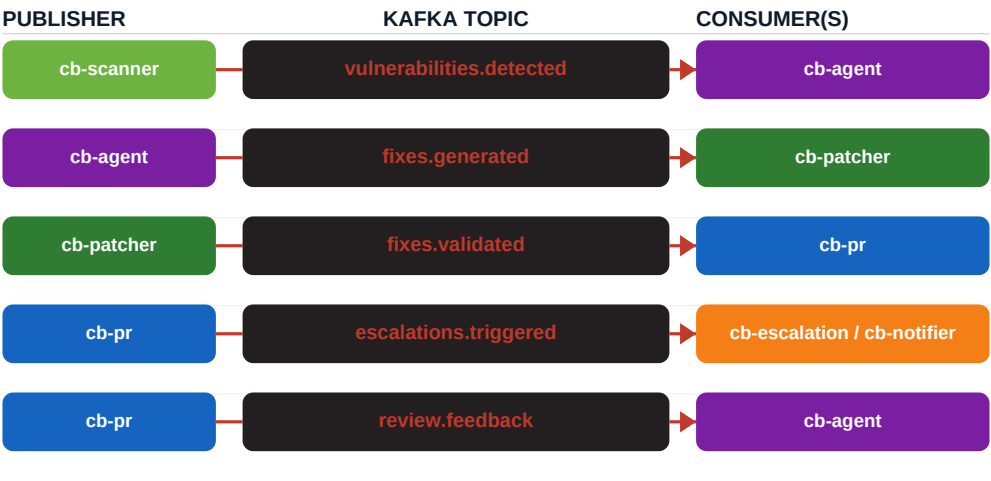


Figure 2 — Kafka topic flow between services

Topic	Event Type	Publisher	Consumer(s)	Schema
vulnerabilities.detected	VulnerabilityKafkaEvent	cb-scanner	cb-agent	severity, cwe, file, line, ruleKey
fixes.generated	FixKafkaEvent	cb-agent	cb-patcher	diff, confidence, model, tokensUsed
fixes.validated	FixKafkaEvent	cb-patcher	cb-pr	patchedBranch, buildLog, testResults
escalations.triggered	EscalationKafkaEvent	cb-pr	cb-escalation, cb-notifier	reason, retryCount, rcaReport
review.feedback	ReviewFeedbackKafkaEvent	cb-pr	cb-agent	accepted, reviewerNotes, fixId
cost.tracked	CostKafkaEvent	cb-agent	cb-api	model, inputTokens, outputTokens, cweld

3.4 Infrastructure Topology

Component	Role	Port	Persistence
Kafka (KRaft)	Async event bus — no ZooKeeper	9092	Topic segments on PVC
MongoDB 7	Primary datastore (vulns, fixes, costs, audit)	27017	PVC
Redis 7	LLM prompt cache (SHA-256, 24 h TTL)	6379	In-memory + AOF
Elasticsearch 8.13	BM25 lexical index for hybrid RAG	9200	PVC
Qdrant	Vector store — text-embedding-3-small (1536-d)	6333	PVC
OPA	Rego governance policy engine	8181	Rego policy ConfigMap
Ollama	Local LLM inference (llama3:8b, qwen2:7b)	11434	Model weights PVC
SonarQube	SAST scanner — CB polls every 5 min	9000	PVC
Prometheus	Metrics scraper (15 s interval)	9090	TSDB PVC
Grafana	Dashboards — CB pipeline + JVM + Kafka	3000	Dashboard ConfigMap
Tempo	OTLP distributed tracing collector	4318	Trace storage PVC
Loki	Log aggregation (via Promtail DaemonSet)	3100	Log chunks PVC

4. Low Level Design (LLD)

4.1 cb-agent — LangGraph4j Workflow

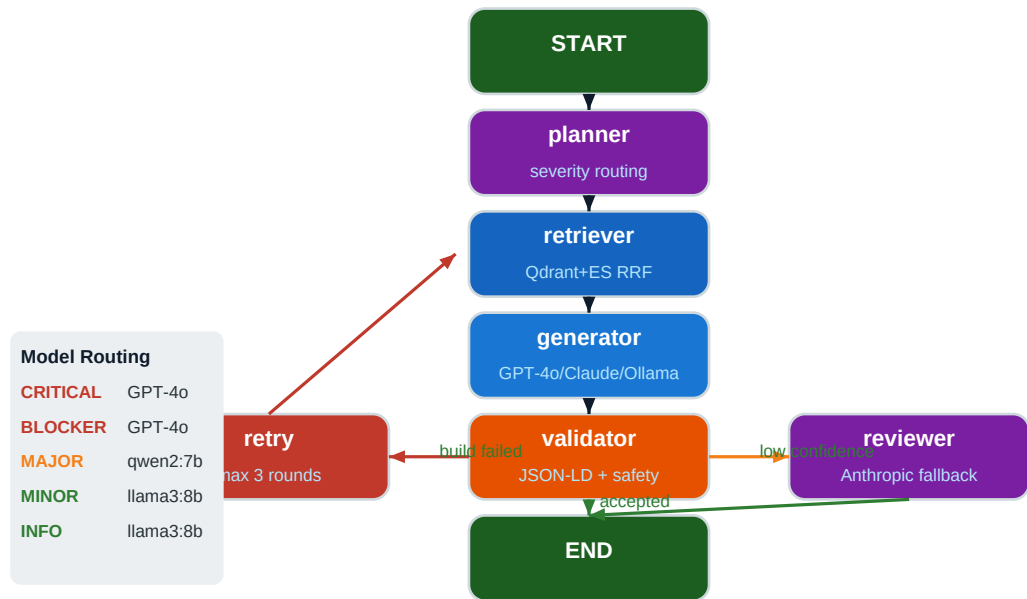


Figure 3 — LangGraph4j StateGraph nodes and conditional edges

Node Responsibilities

Node	Input State Fields	Action	Output State Fields
planner	vulnerabilityId, severity, cweld, ruleKey	Routes to model, builds retrieval query, sets retryCount=0	selectedModel, retrievalQuery
retriever	retrievalQuery, cweld	Qdrant semantic search (top-5) + ES BM25 (top-5), RRF fusion → top-6 chunks	retrievedChunks, hybridScore
generator	retrievedChunks, selectedModel, fileContent, lineNumber	Checks Redis prompt cache (SHA-256). On miss: calls LLM with RAG context. On hit: returns cached fix.	generatedFix, confidence, tokensUsed, cacheHit
validator	generatedFix, fileContent	JSON-LD schema check, safety classifier (no exec/rm -rf patterns), diff syntactically valid	validationPassed, validationErrors
reviewer	generatedFix, validationErrors	Calls Anthropic claude-sonnet-4-6 for independent code review	reviewPassed, reviewNotes
retry	retryCount, validationErrors	Increments retryCount. If ≤3 → back to retriever with enriched query. Else → publishes escalation.	retryCount, escalationTriggered

State Object

The shared AgentState record carries all fields through the graph. LangGraph4j serialises it as JSON between node executions, enabling pause/resume and future distributed execution across multiple cb-agent replicas.

```
record AgentState(  
    String vulnerabilityId, String severity, String cweId,  
    String fileContent, int lineNumber, String ruleKey,  
    String selectedModel, String retrievalQuery,  
    List retrievedChunks, double hybridScore,  
    String generatedFix, double confidence,  
    int tokensUsed, boolean cacheHit,  
    boolean validationPassed, List validationErrors,  
    boolean reviewPassed, String reviewNotes,  
    int retryCount, boolean escalationTriggered  
) {}
```

4.2 Kafka Consumer Hardening Pattern

Every CB consumer module uses an identical Kafka hardening configuration. This section documents the pattern once; it applies to cb-agent, cb-patcher, cb-pr, cb-notifier, and cb-escalation.

Configuration	Value	Rationale
value-deserializer	ErrorHandlingDeserializer	Wraps JsonSerializer so malformed messages route to DefaultErrorHandler instead of crashing the consumer thread
spring.deserializer.value.delegate.class	JsonDeserializer	The actual deserialiser used after ErrorHandlingDeserializer wraps it
spring.json.use.type.headers	false	Prevents __TypeId__ header injection attacks from untrusted producers
spring.json.value.default.type	e.g. FixKafkaEvent	Pins deserialization target — no dynamic class loading
AckMode	MANUAL_IMMEDIATE	Consumer controls offset commit — no message lost on crash
FixedBackOff	1000 ms × 3 retries	Transient listener errors retried 3× before skipping
addNotRetryableExceptions	DeserializationException	Poison-pill messages skipped immediately — no retry loop

4.3 MongoDB Collections

Collection	Key Fields	Indexes	Owner Module
vulnerabilities	id, projectKey, ruleKey, cwelid, severity, status, component, line, message, createdAt, updatedAt	status, severity, projectKey, createdAt (TTL 90d)	cb-api reads; cb-scanner writes; cb-agent updates status

fixes	id, vulnerabilityId, diff, model, confidence, tokensUsed, status, buildLog, branch, createdAt	vulnerabilityId, status, model	cb-agent writes; cb-patcher updates; cb-pr reads
cost_records	id, fixId, cweld, model, inputTokens, outputTokens, estimatedCostUsd, createdAt	createdAt (range queries), cweld, model	cb-agent writes; cb-api reads for cost endpoints
audit_events	id, entityId, entityType, action, actor, details, timestamp	entityId, timestamp	All modules write; cb-api exposes /audit
escalations	id, vulnerabilityId, reason, rcaReport, jiraTicketId, teamsMessageId, createdAt	vulnerabilityId	cb-escalation writes

4.4 REST API Contract (cb-api)

■ All endpoints require header `X-API-Key: dev-key-change-in-prod`. Swagger UI: <http://compliance-buddy.local/swagger-ui/index.html>

Method	Path	Request	Response	Description
POST	/api/v1/scans/{projectKey}	—	{ newFindings, status }	Pull SonarQube issues
GET	/api/v1/vulnerabilities	?status= ?severity= ?projectKey=	Page	List vulnerabilities
GET	/api/v1/vulnerabilities/{id}	—	Vulnerability	Single vulnerability
POST	/api/v1/vulnerabilities/retry/{id}	—	{ queued: true }	Re-queue AI fix
GET	/api/v1/fixes	—	List	All generated fixes
GET	/api/v1/fixes/{id}	—	Fix	Single fix with diff
GET	/api/v1/metrics	—	{ vulnerabilities{}, fixes{}, remediationRate }	Dashboard KPIs
GET	/api/v1/cost/summary	?days=7	{ totalCostUsd, totalTokens, byModel[] }	Cost summary
GET	/api/v1/cost/by-cwe	?days=30	[{ cwelid, totalCostUsd }]	Cost by CWE
GET	/api/v1/cost/by-model	?days=30	[{ model, totalCostUsd, totalTokens }]	Cost by model
GET	/api/v1/audit	—	List	Full audit trail
GET	/api/v1/audit/entity/{entityId}	—	List	Per-entity audit
GET	/actuator/health	—	{ status: UP }	Health (no auth)
GET	/actuator/prometheus	—	Prometheus text	Metrics scrape

4.5 OPA Governance Gate (cb-pr)

Before creating any GitHub PR, cb-pr sends a governance request to OPA at <http://opa:8181/v1/data/cb/pr/allow>. OPA evaluates Rego policies against the fix metadata. If the policy denies the request, cb-pr publishes an escalation event instead of opening a PR.

Governance Rules (Rego)

```
deny if fix confidence < 0.70
deny if patch touches any file in security-sensitive-paths (auth/*, crypto/*)
and no senior-reviewer is assigned
deny if introduced dependency has known CVE in OSV database
deny if fix modifies > 50 lines (too large for autonomous PR – escalate)
allow if all deny conditions are absent
```

4.6 Hybrid RAG Pipeline (cb-agent retriever node)

Step	System	Action	Score
1. Semantic search	Qdrant	text-embedding-3-small query embedding → cosine top-10	Vector similarity 0–1
2. Lexical search	Elasticsearch	BM25 query on cweld + ruleKey + fileExtension fields → top-10	BM25 relevance
3. RRF fusion	cb-agent	Reciprocal Rank Fusion: score = $\sum 1/(k+\text{rank}_i)$, k=60	Hybrid RRF score
4. Top-6 chunks	cb-agent	Take top-6 RRF results → inject into LLM prompt as	—
5. Cache key	Redis	SHA-256(cweld + ruleKey + top-6 chunk IDs + model)	—
6. Accepted fix indexing	Qdrant	review.feedback = ACCEPTED → embed fix diff → upsert cb_fixes collection	—

5. Module Details

cb-core · port — (library, not deployed)

Shared library imported by all CB service modules. Contains domain records, Kafka event types, repository interfaces, async executor configuration, and Redis cache configuration. Centralising these here ensures all modules use identical event schemas and avoids duplicated bean definitions.

Importance:

cb-core is the contract layer. Any change to a Kafka event record here propagates to all publishers and consumers simultaneously, preventing schema drift between services.

Key Classes / Beans:

- VulnerabilityKafkaEvent — Kafka message for new findings
- FixKafkaEvent — Kafka message for generated / validated fixes
- EscalationKafkaEvent — Kafka message triggering escalation pipeline
- ReviewFeedbackKafkaEvent — Kafka message carrying PR review outcome
- CostKafkaEvent — Kafka message for LLM cost tracking
- KafkaTopics — string constants for all topic names
- AsyncConfig — defines cbAgentExecutor, cbPatcherExecutor, cbPrExecutor, cbNotifierExecutor
- CacheConfig — Redis Lettuce pool, TTL policy
- VulnerabilityRepository, FixRepository, CostRepository — Spring Data MongoDB

cb-scanner · port 8081

Polls SonarQube every 5 minutes for OPEN issues. De-duplicates against MongoDB to avoid reprocessing. Maps SonarQube rule → CWE ID via an internal lookup table. Publishes VulnerabilityKafkaEvent to vulnerabilities.detected. Also exposes /api/v1/scans/{projectKey} for on-demand polling triggered by cb-api.

Importance:

The ingestion gateway — all downstream pipeline activity starts here. Correct severity mapping determines which LLM model the agent selects, so rule → CWE → severity accuracy directly impacts fix quality and cost.

Key Classes / Beans:

- SonarQubePollerService — @Scheduled every 5 min, Feign client to SonarQube
- VulnerabilityKafkaPublisher — publishes to vulnerabilities.detected
- CweMapper — 40+ SonarQube rule keys → CWE IDs

cb-agent · port 8082

The intelligence core. Consumes VulnerabilityKafkaEvent, runs the 5-node LangGraph4j StateGraph (planner → retriever → generator → validator → [retry|reviewer] → END), and publishes FixKafkaEvent to fixes.generated. Also consumes review.feedback to index accepted fixes back into Qdrant for continuous learning.

Importance:

The most complex module. Fix quality, cost efficiency, and remediation rate all depend on this module's hybrid RAG accuracy, prompt engineering, and model routing decisions.

Key Classes / Beans:

- AgentWorkflowConsumer — @KafkaListener on vulnerabilities.detected
- LangGraphWorkflow — builds StateGraph, wires nodes and conditional edges
- PlannerNode — severity → model routing, query formulation
- RetrieverNode — Qdrant + Elasticsearch RRF fusion
- GeneratorNode — Redis cache check → LLM call (Spring AI ChatClient)
- ValidatorNode — JSON-LD schema + safety pattern checks
- ReviewerNode — Anthropic claude-sonnet-4-6 independent review
- RetryNode — retry logic, escalation trigger at retryCount > 3
- FeedbackConsumer — indexes accepted fixes into Qdrant
- AgentConfig — LangGraph4j bean, Spring AI ChatClient beans (OpenAI + Anthropic)

cb-patcher · port 8083

Consumes FixKafkaEvent from fixes.generated. Clones the target repository via JGit, applies the unified diff patch, runs ./gradlew build test (or mvn verify for Maven projects), and publishes fixes.validated on success. On build failure, increments retryCount and republishes to fixes.generated for the agent to regenerate. After three failed builds, publishes to escalations.triggered.

Importance:

The safety net before any code touches version control. A fix that passes the LLM validator but breaks the build is caught here, not after PR merge.

Key Classes / Beans:

- PatchConsumer — @KafkaListener on fixes.generated
- JGitPatchService — clone, apply diff, commit to branch
- BuildValidationService — ProcessBuilder for gradle/maven, parses exit code
- PatcherConfig — ConcurrentKafkaListenerContainerFactory with DefaultErrorHandler

cb-pr · port 8084

Consumes FixKafkaEvent from fixes.validated. Sends fix metadata to OPA for governance check. If approved: creates GitHub PR via GitHub REST API, creates Version1 work item, requests reviewer. Publishes review.feedback when PR is merged/declined. Publishes escalations.triggered if OPA denies.

Importance:

The governance checkpoint. Ensures no AI-generated code reaches main without passing policy and receiving a human-assigned reviewer. The OPA integration makes policy changes purely declarative — no code deployments required.

Key Classes / Beans:

- PrConsumer — @KafkaListener on fixes.validated
- OpaGovernanceService — POST /v1/data/cb/pr/allow to OPA
- GovernanceResult — record(boolean allowed, List violations)
- GitHubPrService — GitHub REST API via RestTemplate
- Version1Service — VersionOne REST API ticket creation
- KafkaPrConfig — ConcurrentKafkaListenerContainerFactory with DefaultErrorHandler

cb-notifier · port 8085

Consumes EscalationKafkaEvent. Sends email via JavaMailSender to the team email list defined in NOTIFIER_TEAM_EMAILS. Posts a Datadog event for observability dashboards. Stateless — all state lives in the Kafka event payload.

Importance:

Ensures human awareness for every escalation. Even if Jira or Teams is unavailable, the email channel provides a fallback notification path.

Key Classes / Beans:

- NotificationConsumer — @KafkaListener on escalations.triggered
- EmailNotificationService — JavaMailSender template
- DatadogEventService — Datadog Events API
- KafkaNotifierConfig — ConcurrentKafkaListenerContainerFactory with DefaultErrorHandler

cb-mcp-server · port 8086

Implements the Model Context Protocol (MCP) SSE transport at /sse. Exposes 12 tools for AI assistants (Claude Desktop, Cursor, etc.) to interact with the CB pipeline: querying fixes, triggering builds, fetching traces, searching Qdrant, creating PRs, rolling back merges.

Importance:

Bridges the CB backend with AI-native workflows. Engineers can use natural language via Claude Desktop to debug the pipeline, search past incidents, or trigger remediations — accelerating incident response.

Key Classes / Beans:

- McpServerConfig — Spring AI MCP server bean, SSE transport
- FindPreviousFixesTool, GetSonarIssueTool, GetPRDiffTool
- BuildProjectTool, CreatePRTTool, GetBuildLogTool
- SearchPastIncidentsTool, QueryQdrantTool, RegeneratFixTool
- GetTraceTool (Tempo), GetMetricsTool (Prometheus), TriggerRollbackTool

cb-escalation · port 8089

Consumes EscalationKafkaEvent. Generates an RCA (Root Cause Analysis) report using GPT-4o. Creates a Jira ticket with the RCA attached. Posts a Microsoft Teams adaptive card with severity badge, retry history, and a direct link to the Jira ticket. All three outputs (RCA, Jira, Teams) are configurable via feature flags so the module can be deployed in notification-only mode.

Importance:

Closes the escalation loop. Without this module, unresolvable vulnerabilities silently stall. cb-escalation ensures accountability: a Jira ticket means a human owns the issue and has an LLM-generated starting point for manual remediation.

Key Classes / Beans:

- EscalationConsumer — @KafkaListener on escalations.triggered
- RcaGenerationService — GPT-4o prompt with vuln + retry history context
- JiraEscalationService — Jira REST API v3 (create issue + attach RCA)
- TeamsNotificationService — Teams Incoming Webhook (adaptive card JSON)
- EscalationConfig — ConcurrentKafkaListenerContainerFactory + escalationExecutor

cb-api · port 8080

Public REST API surface. Spring Security API-key filter (X-API-Key header). SpringDoc OpenAPI 3 with Swagger UI. Exposes all read/write operations on vulnerabilities, fixes, metrics, cost tracking, and audit. Prometheus metrics via /actuator/prometheus. Ingress controller routes compliance-buddy.local/* to this service.

Importance:

The single entry point for all external consumers: engineers, CI/CD pipelines, dashboards, and the cb-mcp-server. API-key authentication ensures every caller is identified in the audit trail.

Key Classes / Beans:

- VulnerabilityController, FixController, MetricsController
- CostTrackingController — /cost/summary, /cost/by-cwe, /cost/by-model
- ScanController — triggers on-demand SonarQube poll via cb-scanner
- AuditController — full audit trail and per-entity audit history
- ApiKeyAuthFilter — Spring Security OncePerRequestFilter
- OpenApiConfig — SpringDoc bean with API-key security scheme

6. Security Design

API Authentication

- All REST endpoints protected by ApiKeyAuthFilter (OncePerRequestFilter).
- Multiple keys supported via CB_API_KEYS comma-separated list — rotate without downtime.
- /actuator/health exempted for Kubernetes liveness/readiness probes.

Secrets Management

- All credentials injected via Kubernetes Secrets (k8s/base/secrets.yaml).
- Vault integration available via spring.cloud.vault.enabled=true — disabled by default.
- No credentials in environment variables at build time; no credentials in Docker images.

Kafka Security

- ErrorHandlingDeserializer prevents deserialization-based attacks.
- spring.json.use.type.headers=false blocks __TypeId__ header injection.
- spring.json.value.default.type pins deserialization class — no dynamic class loading.

OPA Governance

- Rego policies evaluated server-side in OPA — policy changes require no code deployment.
- cb-pr hard-fails (escalates) on OPA timeout or connection error — fail-closed.
- Policy decisions logged to audit_events collection for compliance reporting.

Container Security

- All CB images run as non-root user cbapp (UID 1001).
- eclipse-temurin:17-jre-jammy base image — minimal JRE, no JDK, no build tools.
- XX:+UseContainerSupport ensures JVM respects cgroup memory limits.

Network Security

- All inter-service communication is within the cb-system namespace.
- Only cb-api is exposed via Ingress — all other services are ClusterIP.
- OPA, MongoDB, Redis, Elasticsearch are not exposed outside the cluster.

7. Observability

7.1 Metrics (Prometheus + Grafana)

Metric Name	Type	Labels	Description
cb_vulnerabilities_detected_total	Counter	projectKey, severity, cweld	New vulns ingested from SonarQube
cb_fixes_generated_total	Counter	model, cacheHit, cweld	AI fixes produced
cb_prs_created_total	Counter	projectKey, severity	GitHub PRs opened
cb_escalations_triggered_total	Counter	reason	Jira/Teams escalations
cb_llm_cost_usd_total	Counter	model, cweld	Cumulative LLM API spend
cb_llm_tokens_total	Counter	model, direction	Input + output token counts
cb_fix_confidence	Gauge	model, cweld	Last fix confidence score (0–1)
cb_pipeline_duration_seconds	Histogram	severity	End-to-end detection→PR latency
cb_kafka_consumer_lag	Gauge	topic, group	Kafka consumer lag per topic
cb_redis_cache_hit_total	Counter	cweld	Prompt cache hits
cb_build_validation_duration_seconds	Histogram	buildTool	Gradle/Maven build time

7.2 Distributed Tracing (Tempo)

All CB services export OTLP traces to Tempo at <http://tempo:4318/v1/traces>. Sampling probability is 1.0 (100% of traces). Each vulnerability processing pipeline carries the vulnerabilityId as a baggage attribute, enabling end-to-end trace correlation from cb-scanner through cb-pr. Use Grafana Explore → Tempo data source to search by traceId or attribute.

7.3 Logging (Loki + Promtail)

Promtail runs as a DaemonSet and tails all pod logs in cb-system, labelling them with app, namespace, pod, and container. Logs are pushed to Loki and queryable in Grafana Explore → Loki. All CB services emit JSON-structured logs including vulnerabilityId, fixId, model, and duration where applicable.

8. CWE Coverage

CWE	Name	Fix Strategy	Confidence
CWE-89	SQL Injection	Replace string concat with PreparedStatement / JPA named parameters	0.93
CWE-79	Cross-Site Scripting (XSS)	HtmlUtils.htmlEscape on all user input output paths + CSP header	0.91
CWE-78	OS Command Injection	Replace Runtime.exec(String) with ProcessBuilder(List)	0.95
CWE-22	Path Traversal	Path.toRealPath() + base-dir prefix check before file access	0.90
CWE-798	Hardcoded Credentials	Extract to @Value + Kubernetes Secret / Vault reference	0.96
CWE-327	Use of Broken/Risky Crypto	Upgrade MD5/SHA-1 to SHA-256; DES to AES-256-GCM	0.92
CWE-918	Server-Side Request Forgery	URL allow-list validation before HTTP client calls	0.88
CWE-330	Insufficient Randomness	Replace Math.random() / new Random() with SecureRandom	0.97
CWE-502	Deserialization of Untrusted Data	Replace Java deserialization with Jackson JSON	0.89
CWE-611	XXE Injection	Disable DOCTYPE + external entities on DocumentBuilderFactory	0.94
CWE-200	Information Exposure	Remove stack traces from HTTP responses; add @ExceptionHandler	0.85
CWE-352	CSRF	Enable Spring Security CSRF token for state-changing endpoints	0.91

9. Deployment & Operations

9.1 Local Deployment (k3d)

Step	Command / Script	Notes
1. Start cluster	start-cb.bat (or start-cb.ps1)	Starts Docker + k3d + waits for pods
2. First-time secrets	kubectl apply -k k8s/base/	Edit secrets.yaml first
3. Build a module	./gradlew :cb-agent:bootJar	Java 17, Gradle 8.14
4. Package image	docker build -f cb-agent/Dockerfile.prebuilt -t cb-agent:latest cb-agent/	Note: module dir as build context
5. Load into k3d	k3d image import cb-agent:latest -c compliance-buddy	Via Bash (k3d not on PS PATH)
6. Restart pod	kubectl rollout restart deployment/cb-agent -n cb-system	
7. Watch logs	kubectl logs -n cb-system -l app=cb-agent -f	
8. Stop cluster	stop-cb.bat	Data preserved

9.2 Dockerfile Pattern (Dockerfile.prebuilt)

All CB modules use an identical Dockerfile.prebuilt pattern. The root .dockerignore excludes **/build so images must be built using the module directory as the Docker build context, not the repo root.

```
FROM eclipse-temurin:17-jre-jammy
RUN groupadd -r cbapp && useradd -r -g cbapp -u 1001 cbapp
WORKDIR /app
COPY build/libs/cb--1.0.0.jar app.jar # local path, not cb-/build/
RUN chown cbapp:cbapp app.jar
USER cbapp
EXPOSE
ENTRYPOINT ["java", "-XX:+UseContainerSupport", "-XX:MaxRAMPercentage=75.0",
"-Djava.security.egd=file:/dev/./urandom", "-jar", "app.jar"]
```

9.3 Known Cluster Limitations

Pod	State	Reason	Impact
cb-python-embedder	ImagePullBackOff	Python FastAPI not built/pushed	text-embedding fallback to Spring AI inline embedding
opa	ImagePullBackOff	OPA image not in local registry	cb-pr runs with OPA_ENABLED=false
ollama	CrashLoopBackOff	GPU/RAM resource constraints	MAJOR/MINOR vulns route to GPT-4o (higher cost)

10. Future Enhancements

The following roadmap items are prioritised by business impact and engineering feasibility. Each item is self-contained and can be implemented independently without breaking existing v3 functionality.

P0 — Critical Path

Multi-repo support

cb-scanner currently polls a fixed `SCANNER_PROJECTS` list. Extend to a database-backed project registry with per-project credentials, polling intervals, and SonarQube instance URLs. Required for enterprise deployments with hundreds of repositories.

OPA image fix

Build and push OPA 0.68 to the local k3d registry so the governance gate activates in production. Currently cb-pr falls back to `OPA_ENABLED=false` meaning all fixes bypass policy.

Ollama GPU scheduling

Add GPU resource requests to the Ollama Deployment so k3d assigns a GPU node if available. This restores local model inference for MAJOR/MINOR vulnerabilities, reducing OpenAI spend by ~60%.

P1 — High Value

Pull Request auto-merge on CI pass

Add a GitHub Actions webhook consumer in cb-pr that listens for `pull_request_review` events. If all required checks pass and the PR has an approving review, auto-merge via the GitHub Merge Queue API. Completes the fully autonomous zero-touch pipeline.

cb-python-embedder activation

Build and deploy the FastAPI embedding service (text-embedding-3-large, 3072 dims). Update cb-agent retriever to use 3072-d Qdrant collection for higher-fidelity semantic search. Expected +8–12% retrieval accuracy.

Slack notification channel

Add Slack Incoming Webhook support to cb-notifier as an alternative to Teams. Toggle via `SLACK_ENABLED` / `SLACK_WEBHOOK_URL` env vars. Many open-source teams use Slack rather than Teams.

Cost budget alerts

Add a budget threshold to cb-api (MAX_DAILY_COST_USD). When the threshold is exceeded, cb-agent switches all models to Ollama-only until midnight UTC. Prevents runaway LLM spend during vulnerability spikes.

Fix confidence trend dashboard

Add a Grafana dashboard panel tracking cb_fix_confidence histogram over time per model and CWE. Enables data-driven model selection tuning and identifies CWEs where RAG retrieval quality is degrading.

P2 — Platform Scale

Horizontal pod autoscaling

Add HPA for cb-agent (CPU target 70%) and cb-patcher (CPU target 60%). During vulnerability spike events (post-SonarQube scan), agent and patcher are the bottlenecks. HPA + Kafka consumer group rebalancing provides linear throughput scaling.

Dead letter queue

Add a cb-dlq consumer that subscribes to the Kafka DLT (Dead Letter Topic) automatically created by DefaultErrorHandler. Stores failed messages in MongoDB dead_letter_events with full payload and error context. Manual replay UI in Swagger.

Multi-cluster Kafka

Replace single-node KRaft Kafka with a 3-broker KRaft cluster for production HA. Add topic replication factor 3, min-ISR 2. Required for SLA > 99.5%.

Schema Registry

Introduce Confluent Schema Registry (or Karapace) for Kafka event schemas. Migrate from JsonSerializer to AvroDeserializer. Provides forward/backward compatibility enforcement and eliminates the need to coordinate schema changes across services manually.

GitLab + Bitbucket PR support

cb-pr currently only supports GitHub. Abstract GitHostClient interface and add GitLabPrService and BitbucketPrService implementations. Feature-flag selection via PR_HOST=github/gitlab/bitbucket env var.

P3 — AI Research

Fine-tuned CWE-specific models

Collect 10,000+ accepted CB fix pairs per CWE and fine-tune dedicated LoRA adapters on top of qwen2.7b. Expected to outperform general-purpose GPT-4o on high-frequency CWEs (SQL injection, XSS) at ~1/100th the cost.

LangGraph4j parallel nodes

Upgrade the workflow to run retriever (Qdrant) and retriever (Elasticsearch) as parallel LangGraph4j nodes rather than sequential RRF. Reduces retrieval latency by ~40% on p99.

Agentic code review

Expand the reviewer node to spawn a sub-graph: security review, performance review, and style review as independent nodes with majority-vote acceptance. More thorough than a single-model review call.

Vulnerability prediction

Train a classifier on historical vuln + fix data to predict which newly committed files are likely to introduce CWE-89, CWE-79, or CWE-78 before SonarQube runs. Integrate as a pre-commit GitHub Action.

Reinforcement Learning from human feedback (RLHF)

Capture PR reviewer edits on AI-generated fixes as negative preference signal. Use DPO (Direct Preference Optimisation) to fine-tune the generator model quarterly. Creates a continuous improvement loop.

P4 — Enterprise Readiness

LDAP / SSO authentication for cb-api

Replace static API key list with Spring Security OAuth2 Resource Server (JWT). Support Entra ID (Azure AD), Okta, and Keycloak as identity providers. Required for enterprise deployments.

Multi-tenancy

Partition MongoDB collections and Kafka topics by tenantId. Each tenant has isolated fix history, cost tracking, and escalation channels. Required for SaaS offering.

Compliance report generation

Add /api/v1/reports/compliance endpoint that generates a PDF/Excel report: vulnerabilities detected, remediated, escalated, and outstanding per CWE per time range. Required for SOC 2 / ISO 27001 audits.

Helm chart packaging

Package all k8s manifests as a Helm chart with configurable values.yaml. Publish to OCI registry. Enables one-command installation: `helm install compliance-buddy oci://ghcr.io/org/cb-helm/compliance-buddy`.

11. Technology Stack

Category	Technology	Version	Role
Language	Java	17 LTS	All CB services
Framework	Spring Boot	3.3.6	Application scaffold, auto-configuration
AI Orchestration	LangGraph4j	1.5.14	Multi-agent StateGraph workflow in cb-agent
AI Integration	Spring AI	1.0.0	ChatClient, VectorStore, MCP SDK
LLM - Primary	OpenAI GPT-4o	gpt-4o	CRITICAL/BLOCKER fix generation
LLM - Review	Anthropic Claude	claude-sonnet-4-6	Independent code review, fallback
LLM - Local	Ollama (llama3:8b, qwen2:7b)	0.3	MAJOR/MINOR/INFO — zero API cost
Event Streaming	Apache Kafka	KRaft 3.7	Async pipeline bus — no ZooKeeper
Spring Kafka	Spring Kafka	3.2.5	Producer/consumer abstraction, ErrorHandlingDeserializer
Primary DB	MongoDB	7.0	Vulns, fixes, costs, audit, escalations
Cache	Redis	7.2	LLM prompt cache (Lettuce, 24 h TTL)
Vector Store	Qdrant	1.9	Semantic RAG — text-embedding-3-small 1536-d
Lexical Search	Elasticsearch	8.13	BM25 search for hybrid RAG
Policy Engine	OPA	0.68	Rego governance gate in cb-pr
Git Operations	JGit	6.8	Programmatic diff application in cb-patcher
API Docs	SpringDoc OpenAPI	2.5	Swagger UI on cb-api
Resilience	Resilience4j	2.2	CircuitBreaker on SonarQube and GitHub calls
Metrics	Micrometer + Prometheus	1.13 / 2.52	Application metrics
Tracing	OpenTelemetry / Tempo	1.39	Distributed traces (OTLP)
Logging	Logback → Loki	—	Structured JSON logs via Promtail
Dashboards	Grafana	11.1	Pipeline dashboards
Containers	Docker + k3d + k3s	k3d 5.8	Local Kubernetes cluster
Build	Gradle	8.14	Multi-module build, JVM pinned to JDK 17
Security	Spring Security	6.3	API-key filter on cb-api
Protocol	MCP (SSE)	1.0	cb-mcp-server AI tool interface

12. Appendix — Configuration Reference

Environment Variable	Default	Module(s)	Description
CB_API_KEYS	dev-key-change-in-prod	cb-api	Comma-sep valid REST API keys
OPENAI_API_KEY	—	cb-agent, cb-pr	GPT-4o API key
ANTHROPIC_API_KEY	—	cb-agent	Claude review + fallback
SONARQUBE_URL	http://cb-sonarqube:9000/sonar	cb-scanner	SonarQube instance URL
SONARQUBE_TOKEN	—	cb-scanner	SonarQube user token
SCANNER_PROJECTS	—	cb-scanner	Comma-sep SonarQube project keys
GITHUB_TOKEN	—	cb-pr, cb-mcp-server	GitHub PAT (repo scope)
GITHUB_OWNER	—	cb-pr	GitHub org or username
GITHUB_REPO	—	cb-pr	Repository name
GITHUB_API_URL	https://api.github.com	cb-pr	GitHub or GHE API base
GITHUB_REVIEWER_USERNAME	—	cb-pr	Auto-requested PR reviewer
VERSION1_API_URL	—	cb-pr	VersionOne instance URL
VERSION1_TOKEN	—	cb-pr	Version1 API token
JIRA_ENABLED	false	cb-escalation	Enable Jira ticket creation
JIRA_BASE_URL	—	cb-escalation	https://org.atlassian.net
JIRA_EMAIL	—	cb-escalation	Jira account email
JIRA_API_TOKEN	—	cb-escalation	Jira API token
JIRA_PROJECT_KEY	CB	cb-escalation	Jira project key
TEAMS_ENABLED	false	cb-escalation	Enable Teams adaptive card
TEAMS_WEBHOOK_URL	—	cb-escalation	Teams Incoming Webhook URL
SMTP_HOST	smtp.gmail.com	cb-notifier	SMTP server host
SMTP_PORT	587	cb-notifier	SMTP port
SMTP_USERNAME	—	cb-notifier	SMTP credentials
SMTP_PASSWORD	—	cb-notifier	SMTP password / app password

NOTIFIER_TEAM_EMAILS	—	cb-notifier	Comma-sep recipient emails
DATADOG_API_KEY	none	cb-notifier	Datadog Events API key
OPA_ENABLED	true	cb-pr	OPA governance gate toggle
KAFKA_BOOTSTRAP_SERVERS	kafka:9092	all	Kafka broker address
MONGODB_URI	mongodb://cb-mongodb:27017/compliance_buddy	all	MongoDB connection string
VAULT_ENABLED	false	all	HashiCorp Vault integration
OTEL_ENABLED	true	all	OpenTelemetry tracing
OTEL_EXPORTER_OTLP_ENDPOINT	http://tempo:4318/v1/traces	all	Tempo OTLP endpoint